Singly Linked List – Search Element In Sorted List Iterative –Space Complexity and Algorithm.

Program:

```
bool searchElementInSortedListIterative(Node **head, int key)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return false;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            if (current->data == key)
            {
                return true;
            }
            current = current->next;
        }
        return false;
    }
}
….
 case 10:
        {
            int key;
            cout << "Enter the element to search: ";
            cin >> key;

            if (searchElementInSortedListIterative(&head, key))
            {
```

```
            cout << "Element found." << endl;
        }
        else
        {
            cout << "Element not found." << endl;
        }
        break;
    }
```

Space Complexity

Input Space:

A. Parameter – Only Definition

The input space consists of the pointer to the head of the list (Node *** head) and the integer key (int key). Both of these require constant space, i.e., O(1).

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int key` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

- Node **head → one pointer (machine word)
- int key → one integer (machine word)

⇒ Θ(1) = O(1)

B. Full – Data Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

- ***Same two machine words as above (Θ(1)), plus***
  - ***Every node that can be reached from *head when the function is invoked.***

  If the list contains n nodes, they occupy Θ(n) bytes.

  ⇒ Θ(n) = O(n)

Auxiliary Space

Auxiliary space refers to any extra space used by the algorithm that is not part of the input. In this function, the auxiliary space is primarily used for the pointer Node * current, which is used to traverse the list. This pointer also requires constant space, i.e., O(1).

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this iterative search function:

1. **`Node *current`**: The function declares only **one** extra variable, a pointer named `current`, to traverse the list. This pointer occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses this single `current` pointer to move through the list. No new memory is allocated during the search.

Since the function only needs one extra pointer to do its job, regardless of the list's length, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Local variables on the stack:

– Node *current

– a few compiler-generated temporaries

Together they use only constant space.

• No dynamic allocations are performed.

• No recursion, so call-stack usage is constant.

⇒ Θ(1) = O(1)

Algorithm

$SrchElemINSortdListItr(START, KEY)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ False\ and\ EXIT.$

$[When\ List\ is\ not\ empty, then\ search\ KEY]$

$Else$

$Set\ PTR := START \begin{bmatrix} This\ pointer\ PTR\ points\ to\ START\ or\ address\ of\ HEAD\ pointer \\ points\ to\ i.e.\ 1st\ NODE. \end{bmatrix}$

$Repeat\ while\ PTR \neq NULL, then:$

$if\ INFO[PTR] = Key, then:$

$Return\ True\ [If\ `Key`\ is\ found]$

$[End\ of\ If\ structure]$

$Set\ PTR := LINK[PTR][This\ update\ PTR\ make\ PTR\ to\ point\ to\ each\ NODE\ in\ search\ of\ Key]$

$[END\ of\ WHILE\ loop]$

$Return\ False\ [If\ Key\ is\ not\ found\ in\ the\ LIST]$

$[END\ of\ If\ structure]$